

Assignment 8 - SOLUTION

Q1 (4 points)

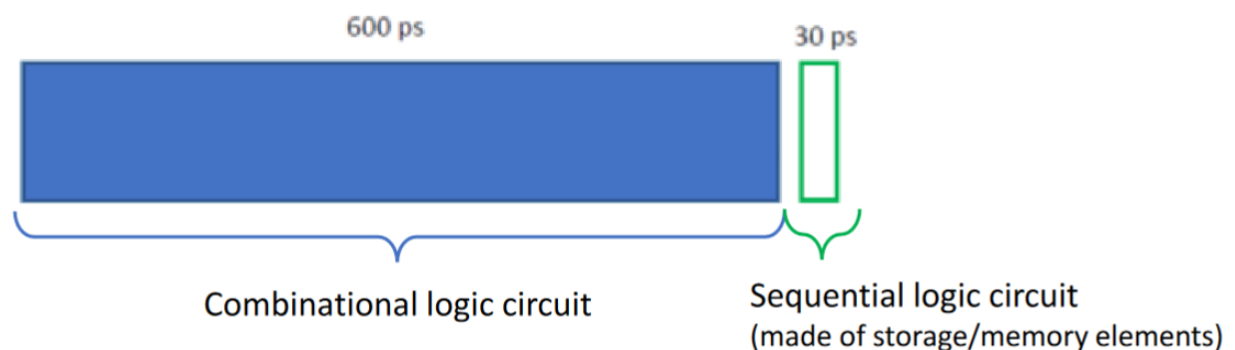
Sequential Instruction Execution and Staged Instruction Execution microprocessor

Part 1

Imagine we design a microprocessor, more specifically, we design its datapath, in such a way that the propagation delay of its entire combinational logic circuit is 600 ps and the propagation delay of its sequential logic circuit (i.e., its storage/memory elements) is 30 ps. Note: Since one *storage/memory element* is capable of remembering one bit of data, you can assume that the sequential logic circuit used in the design of this microprocessor contains enough storage/memory elements to remember all the bits expressing the result of the combinational logic circuit.

[Click here](#) to see the *datapath* of this Sequential Instruction Execution microprocessor.

Sequential Instruction Execution Microprocessor



- What is the minimum clock cycle time (in picoseconds), the latency (in picoseconds), the throughput (in GIPS) and the CPI of our microprocessor if it executes instructions sequentially, i.e., one instruction executed at a time?

Solution:



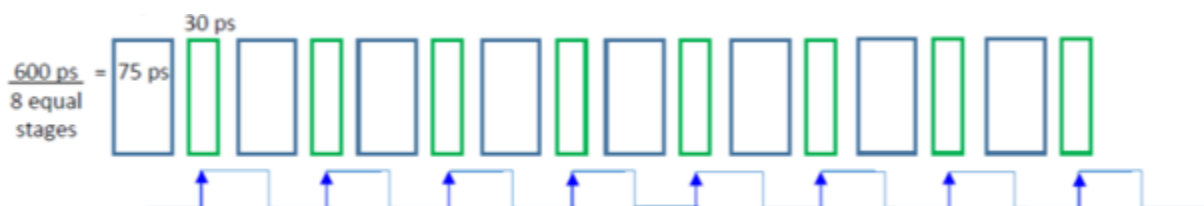
- minimum clock cycle time: $600 \text{ ps} + 30 \text{ ps} = 630 \text{ ps}$
- latency: $600 \text{ ps} + 30 \text{ ps} = 630 \text{ ps}$, 1 instruction takes 630 ps to execute (1 instruction per clock cycle)
- throughput: # of instructions executed per second
 $= 1 \text{ instruction per } 630 \text{ ps} = 1/630 \text{ ps} = 1.5873 \times 10^{-3} \text{ instruction per picosecond}$
 $= 1.5873 \times 10^9 \text{ instruction per second}$
 $= 1.5873 \text{ GIPS}$
- CPI: 1

Part 2

Let's now divide the combinational logic circuit of our Sequential Instruction Execution microprocessor (designed in Part 1 above) into eight (8) equal stages. This new design produces a Staged Instruction Execution microprocessor. Note that this new microprocessor is still executing one instruction at a time. Also, note that we are using the same sequential logical circuit as the one used in the design of our microprocessor in Part 1 above, i.e., with the same latency. Finally, in this question, we are not concerned about naming these eight stages.

- What is the minimum clock cycle time (in picoseconds), the latency (in picoseconds), the throughput (in GIPS) and the CPI of our newly designed Staged Instruction Execution microprocessor?

Solution:



- minimum clock cycle time: $600/8 \text{ ps} + 30 \text{ ps} = 75 \text{ ps} + 30 \text{ ps} = 105 \text{ ps}$
- latency: $(600/8 \text{ ps} + 30 \text{ ps}) 8 = (105 \text{ ps}) 8 = 840 \text{ ps}$, 1 instruction takes 840 ps to execute (1 instruction per 8 clock cycles)
- throughput: # of instructions executed per second
 $= 1 \text{ instruction per } 840 \text{ ps} = 1/840 \text{ ps} = 1.19 \times 10^{-3} \text{ instruction per picosecond}$
 $= 1.19 \times 10^9 \text{ instruction per second}$
 $= 1.19 \text{ GIPS}$
- CPI: 8

Q2 (8 points)

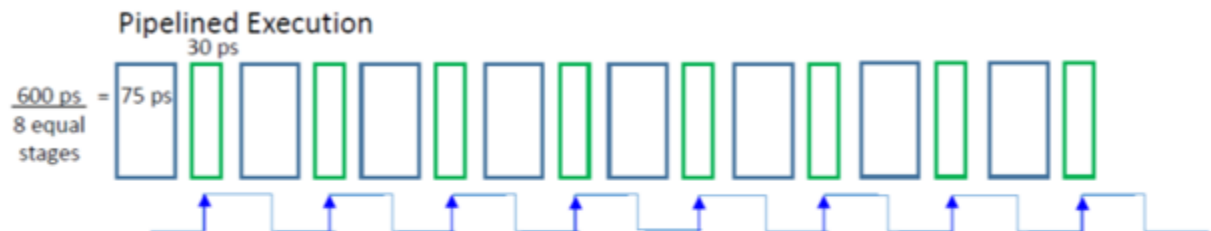
Pipelined Instruction Execution microprocessor (uniform and non-uniform length stages)

Part 1

Consider the Staged Instruction Execution microprocessor we designed in Part 2 of Question 1 above. Imagine now that this new microprocessor executes one instruction at every tick of the system clock. In other words, we have now designed a Pipelined Instruction Execution microprocessor with uniform length stages.

- What is the minimum clock cycle time (in picoseconds), the latency (in picoseconds), the throughput (in GIPS) and the CPI of this new Pipelined Instruction Execution microprocessor?

Solution:



- minimum clock cycle time: $600/8 \text{ ps} + 30 \text{ ps} = 75 \text{ ps} + 30 \text{ ps} = 105 \text{ ps}$
- latency: $(600/8 \text{ ps} + 30 \text{ ps}) 8 = (105 \text{ ps}) 8 = 840 \text{ ps}$, 1 instruction takes 840 ps to execute, i.e., to go through all 8 stages of its execution
- throughput: a fully pipelined processor (at steady state, i.e., when all stages are busy executing micro operations of the machine level instructions) can execute 1 instruction per clock cycle
 $= 1 \text{ instruction per } 105 \text{ ps} = 1/105 \text{ ps} = 9.5238 \times 10^{-3} \text{ instruction per picosecond}$
 $= 9.5238 \times 10^9 \text{ instruction per second}$
 $= 9.5238 \text{ GIPS}$
- CPI: 1

Part 2

Requirement: First, read Section 4.4.3 (subsection on “Nonuniform Partitioning”) of our textbook (Bryant and O'Hallaron) and do its Practice Problem 4.28.

Imagine we design a microprocessor, more specifically, we design its datapath, in such a way that the propagation delay of its entire combinational logic circuit is 600 ps and the propagation delay of its sequential logic circuit (i.e., its storage/memory elements) is 30 ps.

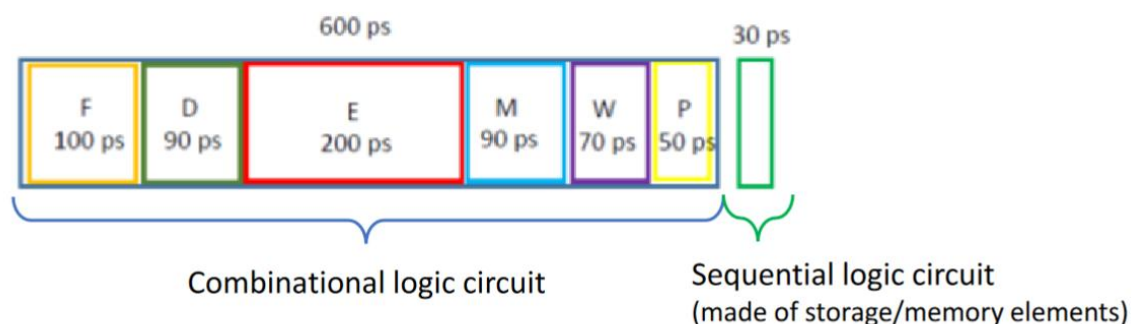
This time, the combinational logic circuit of this microprocessor has been divided into six (6) named stages:

- stage F with a propagation delay of 100 ps,
- stage D with a propagation delay of 90 ps,
- stage E with a propagation delay of 200 ps,
- stage M with a propagation delay of 90 ps,
- stage W with a propagation delay of 70 ps, and
- stage P with a propagation delay of 50 ps.

And so far, no pipeline registers have yet been placed between any of these 6 stages. Let's call the resulting datapath of this microprocessor, as it is described above, its **original datapath**.

[Click here](#) to see this *original datapath*.

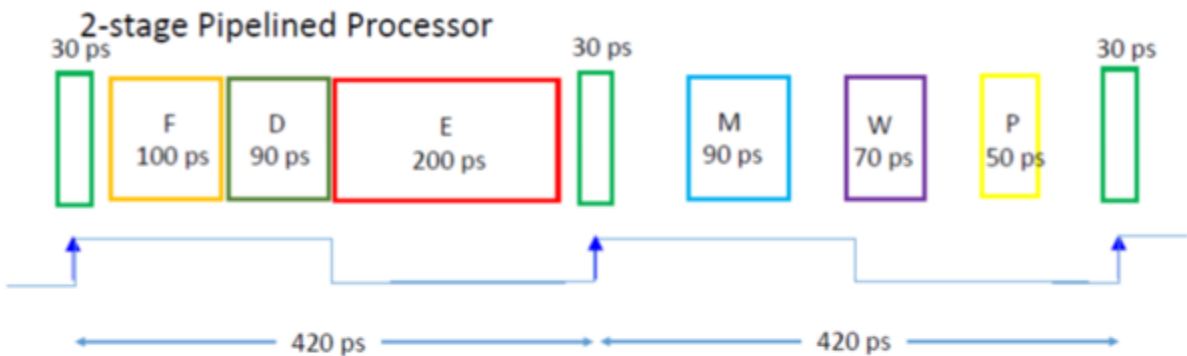
Original Datapath



Note: These six stages cannot be shuffled, i.e., their order is fixed: F – D – E – M – W – P

1. If we were to place a pipeline register between a pair of adjacent stages of this original datapath in order to form a 2-stage pipelined microprocessor, where would we place this pipeline register such that the clock cycle time is minimized? Compute the minimum clock cycle time and the maximum CPU throughput of this 2-stage pipelined microprocessor.

Solution:



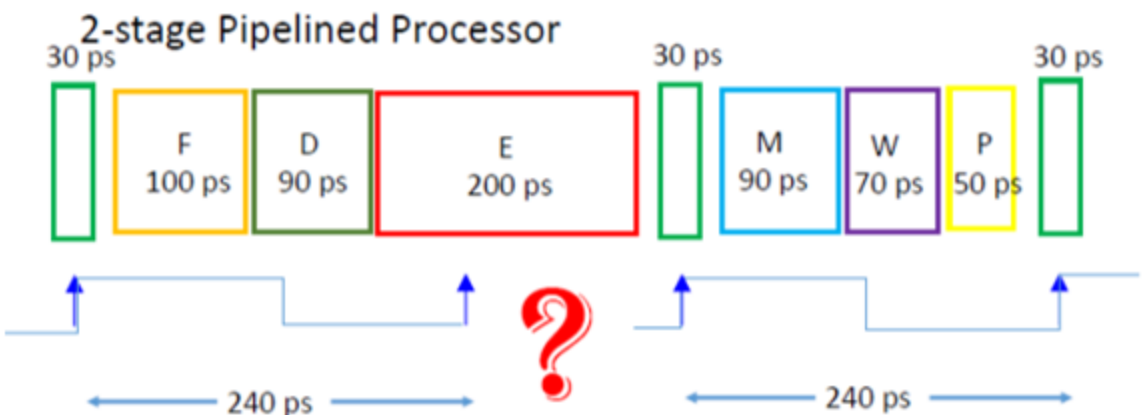
Note: These six stages cannot be shuffled, i.e., their order is fixed: F – D – E – M – W – P

- **Minimum clock cycle time:** $(100+90+200) \text{ ps} + 30 \text{ ps} = 390 \text{ ps} + 30 \text{ ps} = 420 \text{ ps}$
- Why not $(90+70+50) \text{ ps} + 30 \text{ ps} = 240 \text{ ps}$?

Because 240 ps would be too fast (the clock would be ticking too fast). As you can see from the diagram below, the signals would not have time to get through the combinational logic circuit of F, D and E, in time for their output to be ready at the “door” of the clocked (or pipelined) register to their right. When this clocked register would get “clocked”, i.e., activated by the rising edge of the clock cycle, it would not have received the proper input by then, hence it would not “remember” the proper result from F, D and E.

Therefore, the **minimum clock cycle time** must be the largest of the two possible clock cycle times.

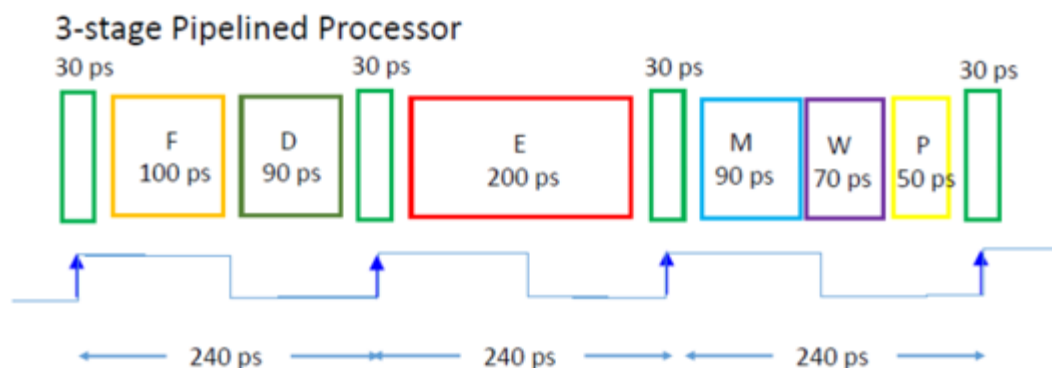
Clock cycle TOO FAST



- Note that other ways of parting the F, D, E, M, W, P stages of the **original datapath** would create a slower clock cycle than 420 ps $\rightarrow$ not good!
 - Here are all the other ways of parting the F, D, E, M, W, P stages of the **original datapath**:
 - F+D (190 ps) and E+M+W+P (410 ps), clock cycle that would work for both stages: $410 \text{ ps} + 30 \text{ ps} = 440 \text{ ps}$
 - F (100 ps) and E+M+W+P (500 ps), clock cycle that would work for both stages: $500 \text{ ps} + 30 \text{ ps} = 530 \text{ ps}$
 - F+D+E+M (480 ps) and W+P (120 ps), clock cycle that would work for both stages: $480 \text{ ps} + 30 \text{ ps} = 510 \text{ ps}$
 - F+D+E+M+W (550 ps) and P (50 ps), clock cycle that would work for both stages: $550 \text{ ps} + 30 \text{ ps} = 580 \text{ ps}$
 - Remember, we are computing the **minimum clock cycle time**, so 420 ps is the minimum of all the possible clock cycle times we can get by parting the **original datapath** in various ways.
 - maximum CPU throughput:** $1/420 \text{ ps} = 2.38 \text{ GIPS}$
- A fully pipelined microprocessor (at steady state, i.e., when the 2 “stages” are busy executing micro operations of the machine level instruction) can execute 1 instruction per clock cycle.

2. If we were to place two pipeline registers between the stages of the original datapath in order to form a 3-stage pipelined microprocessor, where would we place these two pipeline registers such that the clock cycle time is minimized? Compute the minimum clock cycle time and the maximum CPU throughput of this 3-stage pipelined microprocessor.

Solution:



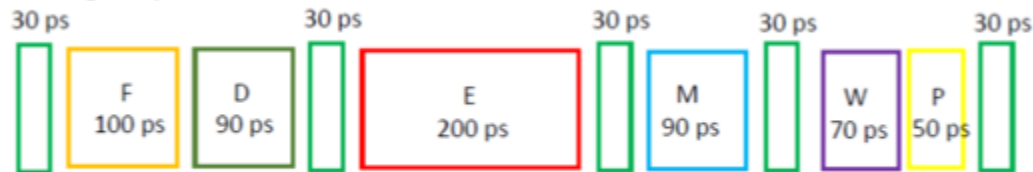
- minimum clock cycle time:** $(90 + 70 + 50) \text{ ps} + 30 \text{ ps} = 210 \text{ ps} + 30 \text{ ps} = 240 \text{ ps}$
 - The other possible clock cycles are too fast:
 - $(100 + 90) \text{ ps} + 30 \text{ ps} = 220 \text{ ps}$
 - $200 \text{ ps} + 30 \text{ ps} = 230 \text{ ps}$
- maximum CPU throughput:** $1/240 \text{ ps} = 4.167 \text{ GIPS}$

3. If we were to place more and more pipeline registers between the stages of the original datapath in order to form greater staged-pipelined microprocessors, which stage would become a *limiting factor* on the minimum clock cycle time?

Hint: Form a 4-stage pipelined microprocessor then a 5-stage pipelined processor and observe what happens. This may help you answer this question.

Solution:

4-stage Pipelined Processor



Minimum clock cycle is 200 ps + 30 ps = 230 ps

5-stage Pipelined Processor



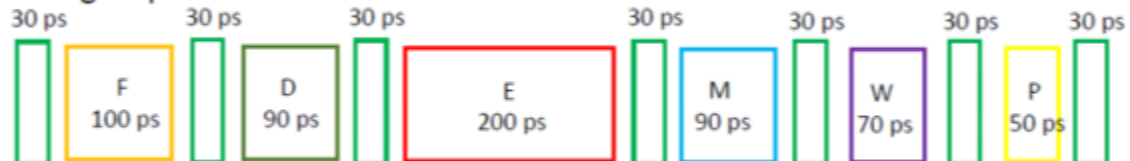
Minimum clock cycle is 200 ps + 30 ps = 230 ps

Solution: The stage E, the longest stage (in ps), is the limiting factor on the minimum clock cycle time since we compute the minimum clock cycle based on the longest stage of the pipelined register.

4. If we were to fully pipeline our original datapath (hence creating a 6-stage pipelined microprocessor), what would its minimum clock cycle time and its maximum CPU throughput be?

Solution:

6-stage Pipelined Processor

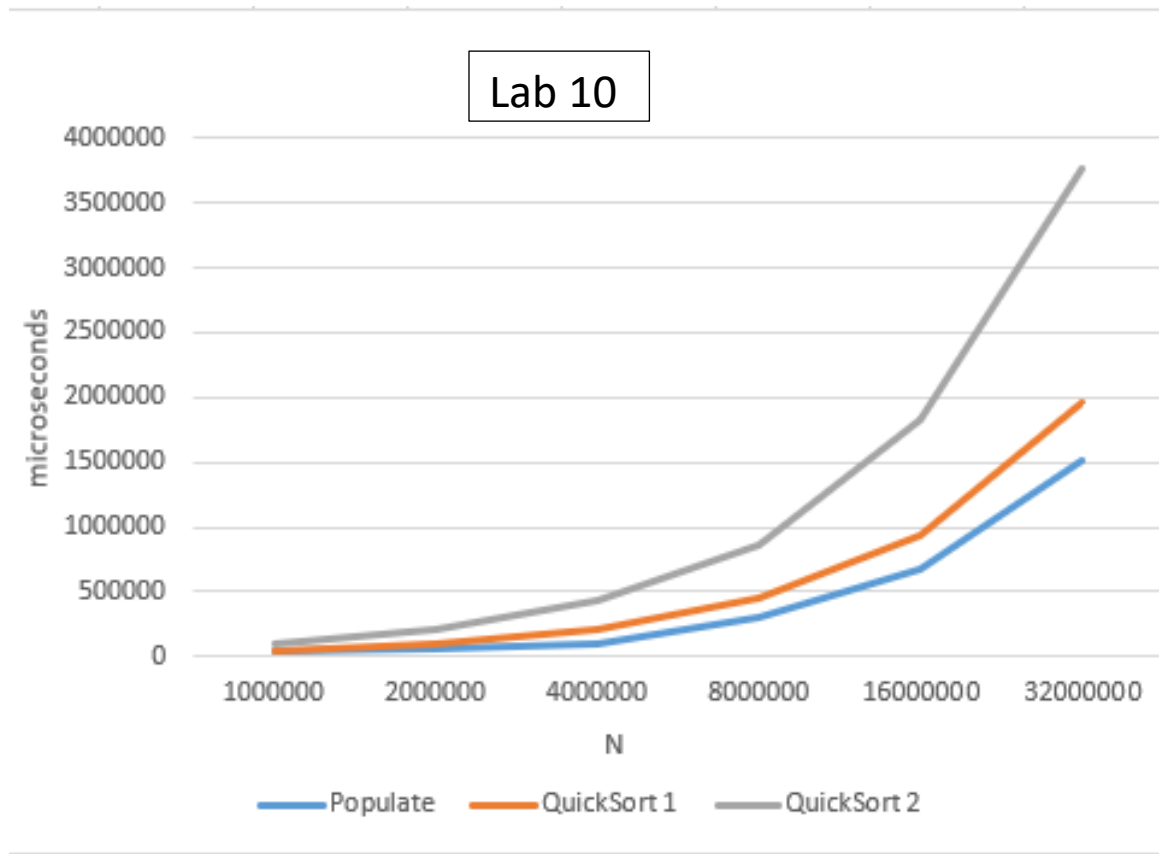


- **minimum clock cycle time:** 200 ps + 30 ps = 230 ps
- **maximum CPU throughput:** 1/230 ps = 4.35 GIPS

Q3 (4 points)

Solution will vary depending on CSIL workstation used!

	A	B	C	D	E	F	G	H
1	Populate	1	2	3	4	5	6	μ
2	1000000	49488	47056	52450	23519	45802	51889	45034
3	2000000	76150	73229	78552	72232	54587	49314	67344
4	4000000	97857	96407	105976	99667	101900	97001	99801.33
5	8000000	312642	287318	290975	298144	304895	302680	299442.3
6	16000000	721089	759574	704826	619927	629285	626195	676816
7	32000000	1508477	1500443	1489173	1475720	1497560	1576051	1507904
8								
9	QSort1-H							
10	1000000	48010	47747	47535	50135	49518	47617	48427
11	2000000	101461	99814	98714	103188	101554	88451	98863.67
12	4000000	211590	208700	213392	215123	214492	220104	213900.2
13	8000000	425325	442047	453114	446919	455118	441223	443957.7
14	16000000	938661	938582	925453	959332	942000	934413	939740.2
15	32000000	1960368	1950993	1976591	1929515	1948972	1974663	1956850
16								
17	QSort2-L							
18	1000000	99759	106436	100647	107165	109513	102493	104335.5
19	2000000	208375	206205	205010	207303	205043	207094	206505
20	4000000	428195	430699	430608	427085	426414	424177	427863
21	8000000	880071	846917	850850	846867	872396	884813	863652.3
22	16000000	1805681	1832526	1795174	1852113	1838215	1809030	1822123
23	32000000	3759916	3780824	3788539	3788055	3671143	3779888	3761394
24								
25		Populate	QuickSort 1	QuickSort 2				
26	1000000	45034	48427	104335.5				
27	2000000	67344	98863.667	206505				
28	4000000	99801.3	213900.17	427863				
29	8000000	299442	443957.67	863652.33				
30	16000000	676816	939740.17	1822123.2				
31	32000000	1507904	1956850.3	3761394.2				



Sensible conclusions:

QuickSort 2 (Lomuto) takes more time to execute than QuickSort 1 (Hoare) and the process of populating the array takes the least amount of time of all three processes.

OR

Populating the array is faster than QuickSort 1 (Hoare) which is faster than QuickSort 2 (Lomuto).

Optional: The fact that QuickSort 1 (Hoare) is faster than QuickSort 2 (Lomuto) was expected.

Q4 (12 points)**Pipeline limitation: Hazards**

Consider the following C code fragment:

```
a = b * e;
c = b + f;
```

Below is the corresponding MIPS assembly code. We are assuming that all C variables are in the stack segment of the memory and that they are all accessible as offsets (displacements) from \$sp:

```
lw $t1, 0($sp) # Loading b from the stack
lw $t2, 4($sp) # Loading e from the stack
mul $t3, $t1, $t2
sw $t3, 12($sp) # Storing a onto the stack
lw $t4, 8($sp) # Loading f from the stack
add $t5, $t1, $t4
sw $t5, 16($sp) # Storing c onto the stack
```

1. The MIPS assembly code above, as it is, does not produce the same result as the C code fragment does. In order to understand why this is and to better visualize what is happening when the above MIPS assembly instructions are executed in each of the 5 stages: fetch (F), decode (D), execute (E), memory (M) and write back (W) of a simple in-order pipelined execution microprocessor, construct the pipelined execution diagram illustrating the pipelined execution of the machine code instructions corresponding to each of these MIPS assembly instructions above. To do so, use the MIPS assembly instructions above as they are, in the order in which they are presented above.

Important: In order to create your pipelined execution diagram, you will need to 1) come up with the micro-operations (a.k.a. micro-instructions) underlying each of the MIPS assembly instructions above. This process is illustrated on Slides 8 and 9 of Lecture 30. Then, 2) following the diagrams found in our Lecture 32, create your pipelined execution diagram. More specifically, these diagrams are found on Slides 8, 15 to 19 in our Lecture 32. These diagrams show the assembly instructions divided into micro-operations listed in the stage in which they occur along with the pipeline registers used: valA, valB, valI, valE, valM.

Solution for 1. Is incorporated in the solution for 2 below.

2. Once you have constructed your pipelined execution diagram, locate the data hazards (data dependencies) in the micro-operations of the illustrated MIPS machine instructions.

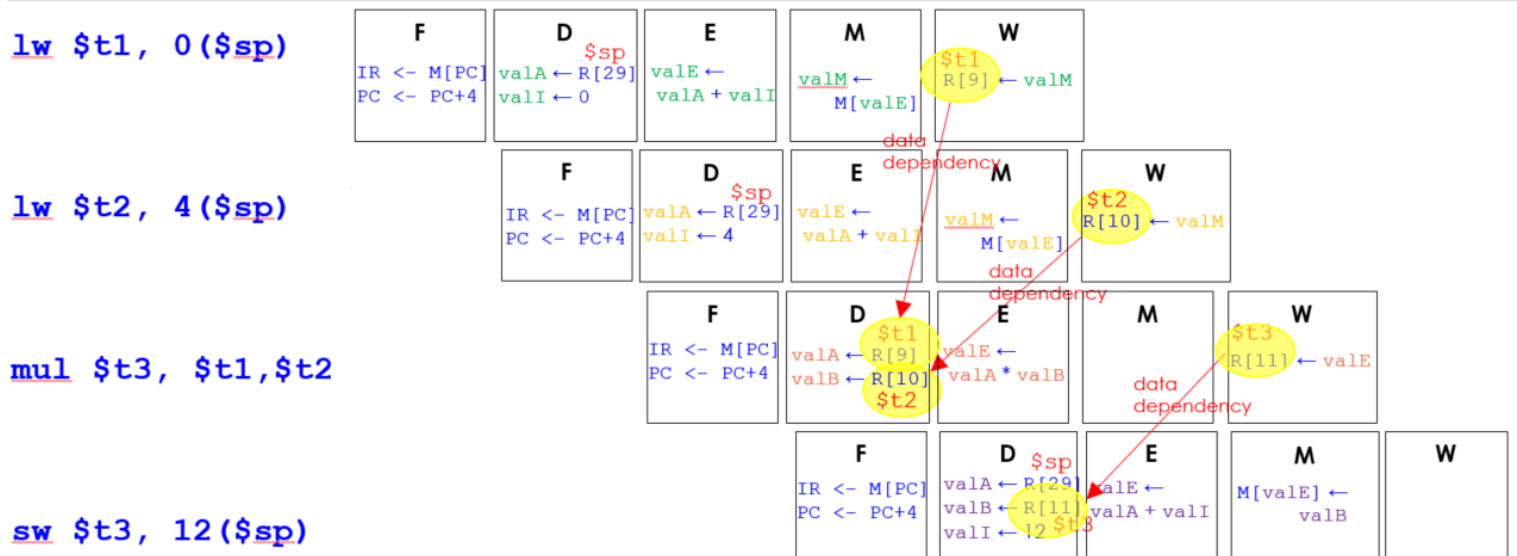
Note: We shall not concern ourselves with structural hazards since MIPS uses two different memories (same as x86-64 ISA):

- one for the instructions and
- one for the data.

So, in MIPS, structural hazards do not create a problem (i.e., it is not a hazard).

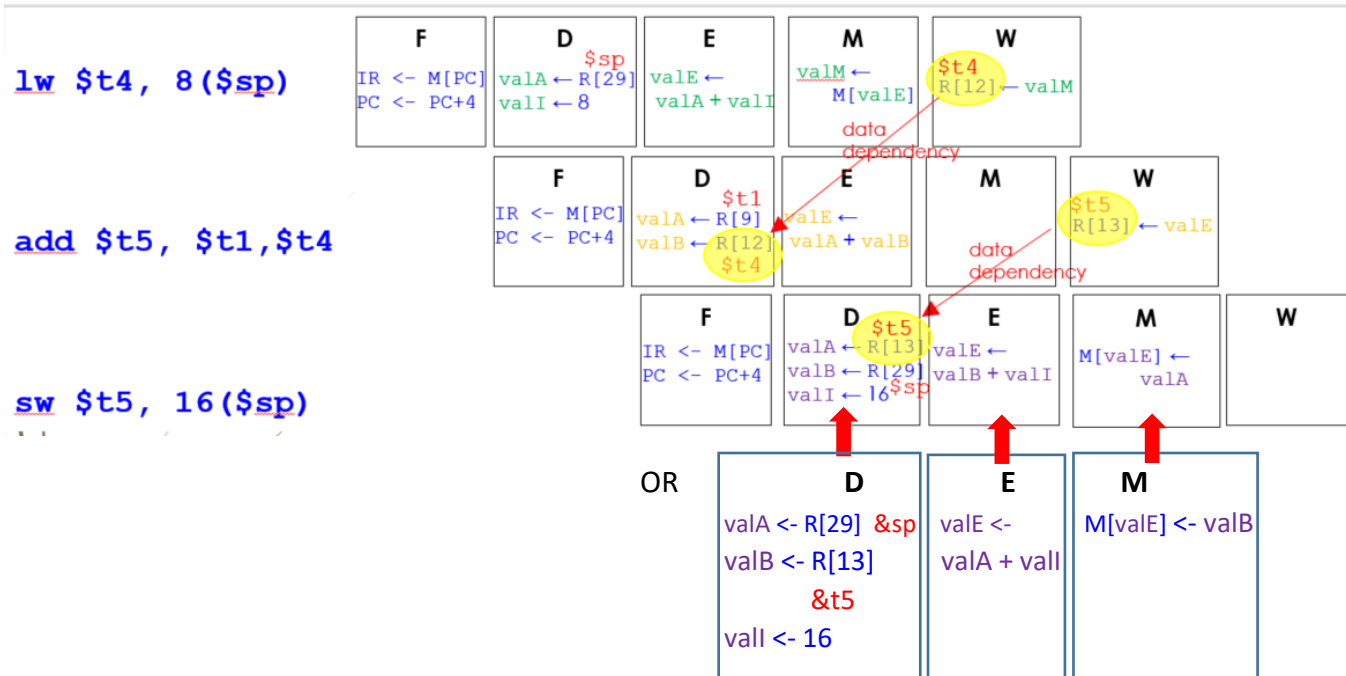
Solution: Below is the **Pipelined Execution Diagram** for 1. and 2. solutions

Data Hazard – There are 5 data dependencies in the given code:



1. `lw $t1, 0($sp)`
2. `lw $t2, 4($sp)`
3. `mul $t3, $t1, $t2`
4. `sw $t3, 12($sp)`

1. Data hazard (creating data dependency) between W stage of instruction 1 (`lw $t1, 0($sp)`) and D stage of instruction 3 (`mul $t3, $t1, $t2`)
2. Data hazard (creating data dependency) between W stage of instruction 2 (`lw $t2, 4($sp)`) and D stage of instruction 3 (`mul $t3, $t1, $t2`)
3. Data hazard (creating data dependency) between W stage of instruction 3 (`mul $t3, $t1, $t2`) and D stage of instruction 4 (`sw $t3, 12($sp)`)



5. `lw $t4, 8($sp)`
6. `add $t5, $t1, $t4`
7. `sw $t5, 16($sp)`

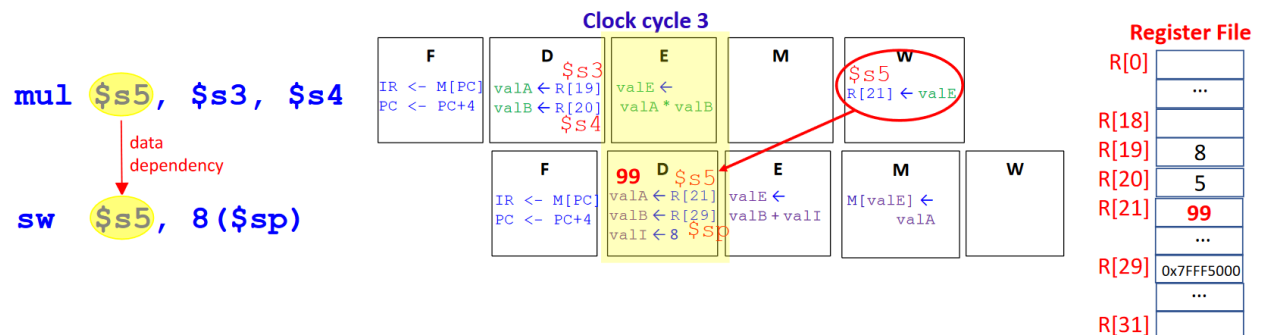
4. Data hazard (creating data dependency) between W stage of instruction 5 (`lw $t4, 8($sp)`) and D stage of instruction 6 (`add $t5, $t1, $t4`)
5. Data hazard (creating data dependency) between W stage of instruction 6 (`add $t5, $t1, $t4`) and D stage of instruction 7 (`sw $t5, 16($sp)`)

Control Hazard

Since there are no *branching* instructions (conditional jumps) in our MIPS assembly code, there are no control hazard (potential problem created by the microprocessor's mis-predictions).

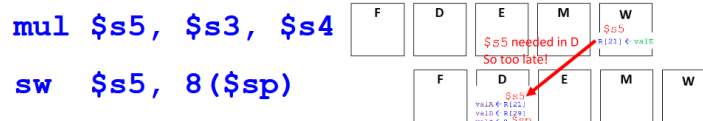
3. Microprocessors fix hazards by using several techniques such as
 - stalling instruction execution by injecting "bubbles" between the instructions that are dependent upon each other,
 - forwarding data,
 - a mix of both (called **load interlock**), and lastly
 - reordering instructions. [Click here for an example.](#)

Data Hazard leading to data dependency



Problem: At Stage D of *sw* (Stage E of *mul*), *sw* needs value of *\$s5*. But *mul* stores value it has computed into *\$s5* only at its Stage W (Stage M of *sw*), i.e., 2 “ticks” later! => **Too late!!!**

Solving data dependency using interleaving



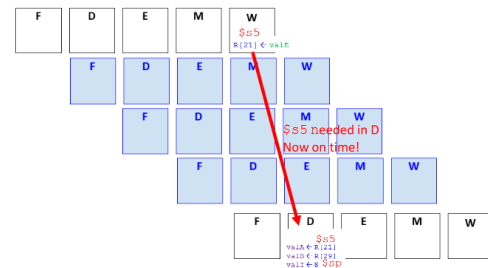
If the original program is:

```
lw $s0, 0($sp)
lw $s1, 4($sp)
...
add $s2, $s6, $s7
...
mul $s5, $s3, $s4
sw $s5, 8($sp)
...
```



I can reorder the instructions, i.e., I can interleave independent instructions between *mul* and *sw*, hence solving their data dependency:

```
mul $s5, $s3, $s4
lw $s0, 0($sp)
lw $s1, 4($sp)
add $s2, $s6, $s7
sw $s5, 8($sp)
```



Fix the data hazards you have discovered by using the reordering instruction (interleaving) technique along with the data forwarding technique. In answering this question, you are not to use the stalling technique nor the load interlock.

As part of your answer, include

- your reordered MIPS assembly instructions (MIPS assembly code) which now should produce the same result as the C code fragment above,
- the corresponding pipelined execution diagram, which must illustrate the new order of your instructions as well as the data being forwarded, and
- clearly indicate on your new pipelined execution diagram, which data is being forwarded and from which stage to which other stage.

Possible solution:

New order:

```

1. lw $t1, 0($sp)
2. lw $t2, 4($sp)
3. lw $t4, 8($sp)
4. mul $t3, $t1,$t2
5. sw $t3, 12($sp)
6. add $t5, $t1,$t4
7. sw $t5, 16($sp)

```

